

Intro to Computer Security — Continuous Project

Part III · Security Domains & Technologies

Week 09 · Lessons 1 & 2

Course Project Context This weekly assignment is one instalment of your **Continuous Course Project**, which you will build incrementally throughout the semester. Each week's deliverable feeds into the next. By the end of the course, the collected weekly reports will form a single, cohesive security portfolio that you will submit as your **Final Assessment**. Review the full project roadmap — including the Midterm Assessment milestone — before you begin, so that each section you write is already written with the final submission in mind.

Legal & Ethical Disclaimer — Read Before You Touch Anything

The tools, techniques, and methods introduced in these lab exercises are intentionally close to those used in real-world professional security work. Their purpose is to give you authentic, hands-on experience in a **controlled, legal environment only**.

You must only perform these exercises on systems, networks, and environments that you own outright, or for which you hold explicit, written authorisation. Applying any of these techniques against systems, networks, or devices belonging to others — without prior consent — is illegal in virtually every jurisdiction and may result in criminal prosecution, civil liability, and permanent academic sanctions.

The instructor and the institution bear no responsibility whatsoever for any misuse of the knowledge or tools presented here. This material is shared strictly for educational purposes. By proceeding, you acknowledge that you accept full personal responsibility for how you apply it.

On tools: Every tool named in these pages is offered as an example only. All named tools are open or freely available for educational use, but you are neither obliged nor restricted to use them. You are actively encouraged to research alternatives, substitute tools you find more appropriate, and — if you have the skill — build your own scripts or utilities. Creativity and independent research are graded as positively as following the suggested path.

Grading Weight for Week 09

Section	Topic	Weight
Part A	Operating Systems (OS) Security and Malware	1
Part B	Network and Web Security	2
Part C	Synthesis, Reflection & Report	1
Total		4 units

The ponderation 1-2-1 means Part B carries double the weight of Parts A and C. Allocate your time accordingly: roughly 1.5 h on Part A, 3 h on Part B, and 1.5 h on Part C (writing and editing the report).

Learning Objectives

By the end of this week, you should be able to:

- Identify common OS (Operating System) attack surfaces and explain how malware exploits them.
 - Classify malware families by behaviour and describe their lifecycle from delivery to execution.
 - Explain the layered architecture of network and web security and trace how a typical attack moves through those layers.
 - Perform basic, legal, hands-on reconnaissance and vulnerability assessment on a system you control.
 - Document findings in a structured, professional security report.
-

What You Will Submit

A single, self-contained PDF or DOCX (Document) report of no more than four (4) A4 pages, including all screenshots, tables, and references. Page count is strict; anything beyond page four will not be read or graded.

Your report must include:

- A brief introduction (3–5 sentences max) contextualising the week's work inside the larger course project.
- All deliverables from Parts A, B, and C, clearly labelled.
- A reference list in any consistent academic style (APA, IEEE, or Chicago).

Formatting reminder: Quality over quantity. Four tight, well-argued pages beat eight rambling ones. Use tables and screenshots economically; every figure must be referenced in the text.

Part A — Operating Systems Security and Malware

Weight: 1 · Estimated time: ~1.5 hours

Background

The OS is the piece of software that mediates every interaction between a user, their applications, and the physical hardware beneath. Because it occupies that privileged position, it is also the most attractive target for attackers. Understanding how the OS enforces (or fails to enforce) security boundaries is therefore foundational to everything else in this course.

Malware (Malicious Software) is the generic term for any software written with the intent to damage, disrupt, or gain unauthorised access to a system. Malware is not a single thing; it is a large family of related but distinct programs — each with its own infection vector (delivery method), persistence mechanism (how it survives a reboot), and payload (what it actually does once running).

Key Malware Families — Quick Reference

Family	Core behaviour	Classic example
Virus	Attaches to a legitimate file; spreads when that file executes	Early boot-sector viruses of the 1980s
Worm	Self-replicates across networks without needing a host file	WannaCry (2017)
Trojan (Trojan Horse)	Disguises itself as benign software	Remote Access Trojans (RATs)
Ransomware	Encrypts victim data and demands payment for the decryption key	LockBit, REvil
Spyware	Silently records and exfiltrates user activity	Keyloggers
Rootkit	Conceals itself and other malware at the OS kernel level	Necurs

Family	Core behaviour	Classic example
Botnet agent	Turns the infected host into a remotely-controlled "bot"	Mirai (IoT-focused)

For authoritative classification guidance, refer to the [MITRE ATT&CK® Framework](#) — a globally recognised knowledge base of adversary tactics and techniques maintained by [MITRE Corporation](#).

Step A-1 — Audit Your OS Baseline (Estimated: 30 min)

Goal: Understand what a "normal" system looks like before you can spot what is abnormal.

Work on a machine you own or a virtual machine (VM (Virtual Machine) — a software-emulated computer running inside your real one). A free hypervisor such as [VirtualBox](#) or [VMware Workstation Player](#) is sufficient. Alternatively, use any live Linux distribution booted from a USB drive.

Procedure

1. List all running processes.

On Linux/macOS, open a terminal and run:

```
ps aux
```

On Windows, open PowerShell and run:

```
Get-Process | Sort-Object CPU -Descending
```

Export or screenshot the output. You will return to this list later.

2. List all services set to start automatically at boot.

On Linux (systemd-based):

```
systemctl list-unit-files --type=service --state=enabled
```

On Windows:

```
Get-Service | Where-Object {$_.StartType -eq 'Automatic'}
```

3. List all active network connections.

```
netstat -tulnp # Linux
```

```
netstat -ano      # Windows
```

Note any ports you do not recognise. Every open port is a potential entry point.

4. Check scheduled tasks.

On Linux:

```
crontab -l && ls /etc/cron.*
```

On Windows:

```
Get-ScheduledTask | Where-Object {$_.State -ne 'Disabled'}
```

Deliverable A-1

In your report, produce a short annotated table (≤ 10 rows) of the *most notable* entries from steps 1–4 — processes, services, ports, or tasks that struck you as interesting, unusual, or that you could not immediately explain. For each entry, write one sentence explaining what you think it does and whether you consider it a potential concern.

Research encouragement: If you encounter a process or service name you do not recognise, look it up on [VirusTotal](#) or cross-reference it with the [National Vulnerability Database \(NVD\)](#) maintained by [NIST \(National Institute of Standards and Technology\)](#). Document your research method — it is part of the grade.

Step A-2 — Static Malware Analysis in a Safe Environment (Estimated: 45 min)

Goal: Examine a known-safe malware sample without executing it. Learn to read what a file *is* before worrying about what it *does*.

Safety rule: Never execute a malware sample on a machine connected to the internet or a network you care about. All analysis in this step is static (no execution). If you choose to go further in your own research, use an air-gapped (fully network-isolated) VM.

Obtaining a Sample for Analysis

Several reputable repositories provide malware samples for research. A good starting point is [MalwareBazaar](#) operated by [Abuse.ch](#). Download only samples marked as "benign for analysis" or labelled as test/decoy files (e.g., the EICAR (European Institute for Computer Antivirus Research) test file, which is universally recognised as safe: <https://www.eicar.org/download-anti-malware-testfile/>).

You are encouraged to find other sample repositories on your own — research is part of the exercise.

Procedure

1. Compute the hash of your sample.

A cryptographic hash (SHA-256 (Secure Hash Algorithm 256-bit) or MD5 (Message Digest 5)) is the malware's "fingerprint". It lets you check whether a file matches a known threat without ever running it.

```
sha256sum filename          # Linux/macOS
Get-FileHash filename       # Windows PowerShell
```

2. Submit the hash to VirusTotal.

Go to <https://www.virustotal.com> and enter your hash. Read and screenshot the results. How many antivirus (AV) engines detected it? What names do they give the threat?

3. Examine the file's strings.

The `strings` utility extracts human-readable text embedded inside a binary. Even without running the file, you can often find URLs, file paths, registry keys, and error messages that reveal what the malware was designed to do.

```
strings filename | grep -E "(http|HKEY|\\Users\\|cmd|powershell)"
```

Alternatives: [FLOSS](#) by [Mandiant](#) automatically de-obfuscates strings. You are encouraged to explore it.

4. Check the file's headers.

Every executable file begins with a short header that identifies what type of program it is. On Linux, `file filename` will tell you the type. On Windows, a free hex editor such as [HxD](#) lets you read the PE (Portable Executable) header directly. Note the compilation timestamp and imported libraries (DLLs (Dynamic-Link Libraries)).

Deliverable A-2

In your report: one paragraph describing what you found — the hash value, the VirusTotal score, two or three interesting strings you extracted, and what you infer about the sample's intended behaviour. Attach one screenshot of the VirusTotal result.

Part B — Network and Web Security

Weight: 2 · Estimated time: ~3 hours

Background

Networks are the arteries of modern computing: they also happen to be the primary channel through which attackers move. Network security is the discipline of protecting data as it travels across those arteries, while web security focuses specifically on the application layer — the HTTP (Hypertext Transfer Protocol) and HTTPS (Hypertext Transfer Protocol Secure) traffic that browsers and servers exchange.

A useful mental model is the **OSI model** (Open Systems Interconnection model) — a seven-layer conceptual framework that describes how data moves from one machine to another. Attacks can target any layer:

OSI Layer	Name	Example attack
7	Application	SQL (Structured Query Language) injection, XSS (Cross-Site Scripting)
6	Presentation	SSL (Secure Sockets Layer) stripping
5	Session	Session hijacking
4	Transport	SYN (Synchronise) flood (DoS (Denial of Service))
3	Network	IP (Internet Protocol) spoofing
2	Data Link	ARP (Address Resolution Protocol) poisoning
1	Physical	Cable tap

For comprehensive standards and guidance on network security, refer to [ISO/IEC 27033](#) published by [ISO \(International Organization for Standardization\)](#) and [IEC \(International Electrotechnical Commission\)](#).

Step B-1 — Network Reconnaissance on Your Own Infrastructure (Estimated: 60 min)

Goal: Experience what an attacker's initial information-gathering phase looks like, using only your own machine or a VM.

Reminder: Run all scans against `localhost` (your own machine, IP address 127.0.0.1) or against a VM on a private, isolated network (e.g., VirtualBox host-only adapter). Scanning any external host without permission is illegal.

Procedure

1. Run a port scan.

[Nmap \(Network Mapper\)](#) is the industry-standard open-source port scanner, used by penetration testers and defenders alike. An alternative is [Masscan](#) — research it and decide which fits your goals better, or create your own basic scanner in Python using the `socket` library.

```
nmap -sV -O -T4 127.0.0.1
```

Flags explained: `-sV` detects service versions, `-O` attempts OS fingerprinting, `-T4` sets an aggressive (but not reckless) timing template.

Screenshot the output. Note: (a) which ports are open, (b) what service Nmap detected on each, and (c) the OS guess.

2. Capture live traffic (optional, advanced).

[Wireshark](#) is an open-source network protocol analyser. Open it, start a capture on your loopback (lo) interface, open a browser, load a plain HTTP page (not HTTPS), and then stop the capture. Apply the display filter `http` and examine the HTTP GET requests. Can you see the request headers? The server's response code?

An alternative or complement: [tcpdump](#) does the same thing from the command line and is pre-installed on most Linux distributions.

3. Check your own firewall rules.

```
sudo iptables -L -v -n          # Linux
netsh advfirewall show allprofiles # Windows
```

Are there rules you cannot explain? Are there ports that *should* be blocked but are not?

Deliverable B-1

Produce a structured table in your report listing each open port found, the detected service, the version (if identified), and a one-sentence risk commentary (e.g., "Port 22 — SSH (Secure Shell) — low risk if properly configured with key-based authentication only; high risk if password authentication is still enabled"). Also include one Wireshark screenshot if you completed the traffic-capture step.

Step B-2 — Web Application Vulnerability Analysis (Estimated: 60 min)

Goal: Identify common web application vulnerabilities in a completely legal practice environment.

Setting Up a Legal Target

Do **not** test any live website. Instead, use one of the following intentionally vulnerable practice applications, all of which are designed to be attacked:

- [DVWA \(Damn Vulnerable Web Application\)](#) — a PHP (PHP: Hypertext Preprocessor)/MySQL-based web app full of deliberate flaws.
- [WebGoat](#) — maintained by [OWASP \(Open Web Application Security Project\)](#).
- [Juice Shop](#) — also by OWASP, modern JavaScript-based target.

Install one (or research and find another you prefer) and run it locally.

Procedure

1. Explore the OWASP Top 10.

The [OWASP Top 10](#) is the most widely referenced list of critical web application security risks. Read the current edition. Pick **two** vulnerability categories from it that you will attempt to demonstrate in your chosen practice application.

2. Attempt a basic SQL injection.

In DVWA, navigate to the SQL injection module. Set the security level to "Low". Enter the classic payload:

```
' OR '1'='1
```

into the user ID field. Observe what happens. Why does this work? What does it reveal about how the query is constructed on the server side?

You may also try tools such as [sqlmap](#) — an open-source automated SQL injection detection and exploitation tool — or build your own simple fuzzer in Python. Research alternatives; document what you chose and why.

3. Attempt a reflected XSS (Cross-Site Scripting) attack.

In DVWA, navigate to the XSS (Reflected) module. Try inserting:

```
<script>alert('XSS proof of concept')</script>
```

into the input field. If the page renders the script rather than escaping it, the vulnerability is confirmed. Take a screenshot of the browser alert box.

Understand *why* this is dangerous: in a real-world scenario the injected script runs in the victim's browser session, potentially stealing cookies, credentials, or performing actions on behalf of the user.

4. Review HTTP security headers.

Open your browser's developer tools (F12), load any page from your practice app, go to the Network tab, click on the main document request, and look at the response headers. Are any of the following missing?

Header	Purpose
Content-Security-Policy (CSP)	Restricts which scripts the page may load
X-Content-Type-Options	Prevents MIME (Multipurpose Internet Mail Extensions) type sniffing
Strict-Transport-Security (HSTS)	Forces HTTPS connections
X-Frame-Options	Prevents clickjacking via iframes

A free online tool that checks headers on public sites is securityheaders.com. Use it to compare a well-configured site against your practice app.

Deliverable B-2

In your report: (a) a paragraph describing your SQL injection test — what payload you used, what the server returned, and why the vulnerability exists; (b) a screenshot of your XSS proof of concept; (c) a small table of the HTTP headers present and absent in your practice app, with a one-sentence explanation of the risk each missing header represents.

Step B-3 — Propose Mitigations (Estimated: 45 min)

Goal: Move from "finding problems" to "recommending solutions" — the mindset of a security analyst, not just an attacker.

Procedure

For each vulnerability you demonstrated in Step B-2, propose a concrete mitigation. Use the following framework:

1. **Describe the root cause** — why does this vulnerability exist?
2. **Propose a technical fix** — a code change, configuration change, or architectural change.
3. **Cite a standard or guideline** that supports your recommendation.

Useful references:

- [OWASP Cheat Sheet Series](#) — practical implementation guidance for dozens of common vulnerability classes.
- [NIST SP 800-53](#) (Special Publication 800-53, Revision 5) — security and privacy controls for information systems, published by [NIST \(National Institute of Standards and Technology\)](#).
- [CWE \(Common Weakness Enumeration\)](#) — a community-developed list of software and hardware weakness types, maintained by [MITRE](#).

Deliverable B-3

In your report: for each of the two OWASP Top 10 vulnerabilities you explored, write a short mitigation section (4–6 sentences each) following the root cause → fix → standard structure above. Be specific; "use better security" is not a mitigation.

Part C — Synthesis, Reflection & Report Assembly

Weight: 1 · Estimated time: ~1.5 hours (including writing)

Step C-1 — Connect This Week to the Bigger Picture (Estimated: 20 min)

Your continuous project is meant to grow into a coherent security portfolio, not a collection of isolated exercises. Before you start writing your final draft, take ten minutes to re-read your deliverables from previous weeks (Weeks 01–08 if available) and ask yourself:

- Do the OS vulnerabilities you found this week relate to the attack techniques you studied in Part I (Threats, Attacks & Defenses)?
- How do the network vulnerabilities you found connect to concepts from earlier sessions on detection, containment, and response?
- What patterns are emerging across the weeks? Is there a common thread — a class of weakness that keeps reappearing?

Write **one paragraph** (6–8 sentences) in your report answering these questions. This is the connecting tissue of your portfolio.

Step C-2 — Write and Assemble the Report (Estimated: 70 min)

Assemble the following sections into a single document, **not exceeding four (4) A4 pages**:

1. **Title block** — Course name, student name(s), student ID(s), week number, submission date.
2. **Introduction** (3–5 sentences) — What is this report about, and how does it fit the larger project?
3. **Part A findings** — Deliverables A-1 and A-2.
4. **Part B findings** — Deliverables B-1, B-2, and B-3.
5. **Synthesis paragraph** — Deliverable C-1.
6. **Conclusion** (2–3 sentences) — What is the single most important thing you learned this week?
7. **References** — All tools, papers, websites, and standards you consulted.

Style Notes

- Write in the third person or first person consistently — do not switch.
 - Every screenshot must be captioned and referenced in the text ("as seen in Figure 1...").
 - Every claim about a vulnerability's impact must be supported by at least one cited source.
 - Proof-read before submitting. Spelling and grammar are part of the grade.
-

Step C-3 — Peer Challenge (Optional, for extra credit)

Exchange reports with a classmate before the submission deadline. Each of you reads the other's vulnerability findings and writes a **one-paragraph rebuttal or extension**: either arguing that a proposed mitigation is incomplete, or suggesting an additional attack vector the author missed. Attach the rebuttal (signed with your classmate's name and ID) as an appendix — it does not count toward the four-page limit.

Submission Checklist

Before you submit, verify each item:

- ☐ Report is exactly 1–4 A4 pages (including all figures and tables).
- ☐ Every tool used is identified by name and a URL to its source.
- ☐ Every vulnerability demonstrated was on a system you own or a deliberately vulnerable practice application.
- ☐ Deliverables A-1, A-2, B-1, B-2, B-3, and C-1 are all clearly labelled in the report.
- ☐ At least three external references are cited.
- ☐ The ethical/legal disclaimer has been read and its principles were respected throughout the exercise.

End of Week 09 Project — Part III · Security Domains & Technologies This document is part of a continuous course project. Retain all previous weekly reports; they will be compiled into the Final Assessment portfolio.